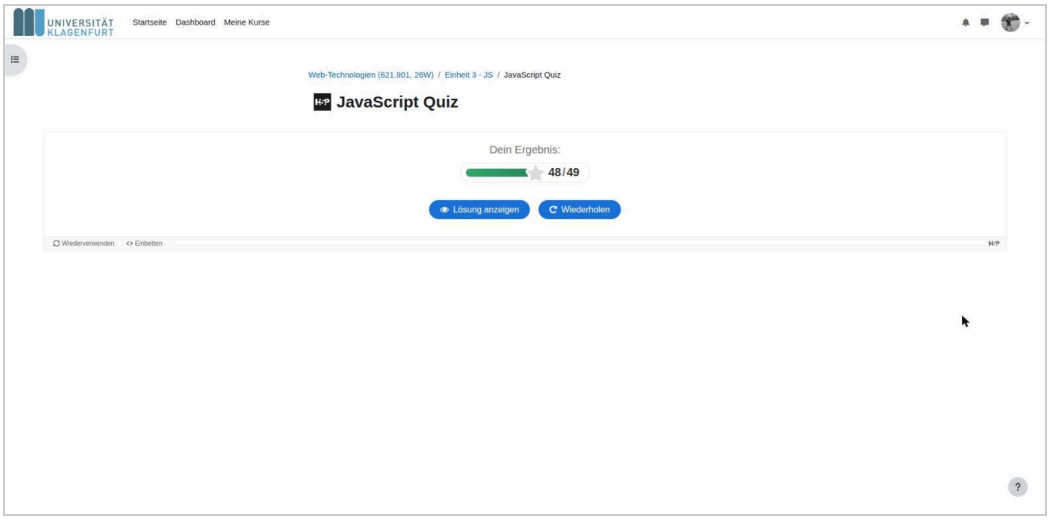
Web-Technologien UE - Assignment 2

Raphael Bleier, WS 2026

Exercise 1 - Quiz

48 von 49 Punkten:



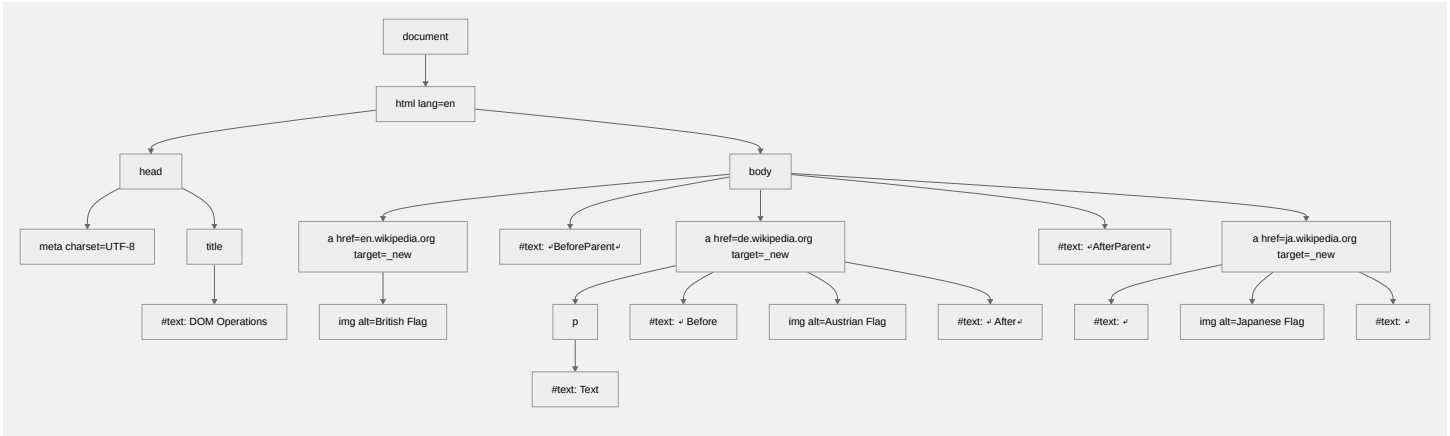
Exercise 2 - Number-Guessing Game I

Siehe `ex2/index.html` und `ex2/game.js`. Die ungültigen Eingaben landen laut Angabe auch im Array, bei "previous guesses" zeig ich aber nur die echten Zahlen an, sonst steht da dreimal die Fehlermeldung drin.

Exercise 3 - DOM Operations

1. DOM von ex3.html

#text = Textknoten, "↵" = Zeilenumbruch. Die reinen Whitespace-Knoten zwischen den Tags sind auch echte Knoten im DOM, deshalb hab ich sie eingezeichnet.



Weggelassen: die Whitespace-Textknoten direkt in head und zwischen body und den Links, sonst wird der Baum unlesbar.

2. Ausgabe erklärt

`getElementsByTagName("a")[1]` gibt den zweiten Link zurück (Index startet bei 0), also den deutschen Wikipedia-Link mit p, Text und österreichischer Flagge.

Zeile	Ausgabe	Warum
`attributes["href"].nodeValue`	https://de.wikipedia.org	Holt den Attribut-Knoten href aus der Attributliste, nodeValue ist der Wert genau so wie er im HTML steht.
`getAttributeNode("href").nodeValue`	https://de.wikipedia.org	Gleich wie davor, nur über die Methode statt über die Liste.
`link.href`	https://de.wikipedia.org/	Die Property gibt die fertig aufgelöste, absolute URL zurück. Deshalb hängt der Browser den Slash an. Bei einem relativen Link würde man hier die ganze URL sehen.
`link.firstChild`	Text	Erstes Kind vom a ist das p, weil zwischen <a ...> und kein Leerzeichen steht. Sonst wär es ein Textknoten.
`attributes[0].nodeType`	2	attributes[0] ist href, Typ 2 = ATTRIBUTE_NODE.
`firstChild.nodeType`	1	Das p ist ein Element, Typ 1 = ELEMENT_NODE.
`parentNode.nodeName`	BODY	Der Link hängt direkt im body. nodeName ist bei HTML Elementen immer groß geschrieben.
`lastChild.parentNode.nodeName`	A	lastChild ist der Textknoten "After", sein Elternknoten ist wieder der Link selbst.
`attributes.length`	2	href und target.
`innerHTML`	Text Before <img ...> After	Der HTML-Inhalt im Link als String, inklusive Zeilenumbrüchen und Einrückung, ohne das a selbst.
`outerHTML`	...	Wie innerHTML, aber das a-Tag selbst ist mit dabei.
`innerText`	"Text↵↵Before After"	Nur der sichtbare Text, so wie er gerendert wird. Das p ist ein Block, deshalb die Zeilenumbrüche danach. Mehrfache Leerzeichen und Einrückungen werden

		zusammengefasst, das Bild ist nicht drin.
`outerText`	gleich wie innerText	Beim Lesen gibt es keinen Unterschied. Erst beim Setzen ersetzt outerText das ganze Element und nicht nur den Inhalt.
`textContent`	"Text↵ Before ↵ After↵"	Der Text aller Textknoten roh hintereinander, mit allen Leerzeichen und Umbrüchen aus dem Quellcode, egal ob sichtbar oder nicht.
`console.log(link)`	das a-Element	Chrome zeigt das Element interaktiv an, man kann es aufklappen und es wird auf der Seite markiert.
`link.nodeValue`	null	nodeValue gibt es nur bei Text-, Kommentar- und Attributknoten. Bei Elementen ist es immer null.

3. removeAttribute("target")

`item(2)` ist der dritte Link (japanische Flagge). Dem wird das target-Attribut weggenommen. Vorher hat er mit `target="_new"` in einem neuen Tab/Fenster geöffnet, danach öffnet er ganz normal im selben Tab. Man sieht davon nichts bis man draufklickt. Im HTML-File ändert sich nichts, nur im DOM im Browser.

Exercise 4 - Glass Price

Siehe `ex4/calculate.js`, das HTML ist unverändert. Die Dicke ist in mm angegeben, deshalb wird sie durch 10 geteilt, damit alles in cm ist. Grau ist die Farbe, wenn die drei Hex-Teile von R, G und B gleich sind, z.B. #000000 oder #808080. Beispiel: 100 x 50 cm, 8 mm, schwarz ergibt 100 · 50 · 0,8 / 50 = 80. In rot sind es 125. Wenn ein Feld fehlt oder ungültig ist (z.B. Breite 300), steht im Preisfeld "-".

Exercise 5 - Event Listeners

1. + 2. Erwartung und Ausgabe

Ich hab erwartet, dass beide Listener laufen, weil der Klick vom Button nach oben zum div bubbelt. Genau so kommt es auch:

```
inner:
target: [object HTMLButtonElement]
currentTarget: [object HTMLButtonElement]
this: [object HTMLButtonElement]
outer:
target: [object HTMLButtonElement]
currentTarget: [object HTMLDivElement]
this: [object HTMLDivElement]
```

Klickt man nur auf das blaue Feld neben dem Button, kommt nur "outer" und target ist dann auch das div.

3. target, currentTarget, this

`target` ist das Element, auf das wirklich geklickt wurde, also wo das Event entstanden ist. Das bleibt während dem ganzen Bubbling gleich. `currentTarget` ist das Element, an dem der Listener hängt, der gerade ausgeführt wird. Beim outer-Listener ist das das div, obwohl der Button geklickt wurde. `this` ist in einer normalen `function` als Event Handler dasselbe wie currentTarget. Bei einer Arrow Function wär das nicht so, da kommt this von außen (hier window).

4. Nur der Button

Ich hab in ex5.js einen zweiten Listener am Button dazugegeben, der `event.stopPropagation()` aufruft. Damit wird das Bubbling nach dem Button gestoppt und der Listener am outerArea wird nicht mehr ausgelöst. Der bestehende Code bleibt so wie er ist.

Exercise 6 - Image Gallery

HTML: Ein div `.gallery` mit den kleinen Bildern. Jedes img hat in `data-large` den Dateinamen der großen Version und im alt die Beschreibung. Darunter das div `#displayDIV` mit einem leeren img `.bigIMG` und einem p `.desc`. Das div ist per CSS versteckt und liegt mit `position: absolute` und 100vw/100vh über der ganzen Seite.

gallery.js: Jedes Galeriebild bekommt einen click-Listener `showImg`. Der setzt die src vom großen Bild auf `e.target.dataset.large`, schreibt den alt-Text in die Beschreibung und macht das displayDIV sichtbar. Ein Klick irgendwo aufs displayDIV ruft `hideImg` auf und versteckt es wieder (über currentTarget, damit es auch beim Klick aufs Bild selbst das div erwischt).

Erweiterung: Zwei Buttons previous/next werden per JS ins displayDIV gehängt. Der aktuelle Index wird gesucht, indem die src vom großen Bild mit den data-large Werten verglichen wird. Damit funktioniert das mit beliebig vielen Bildern. Am Anfang springt previous ans Ende und am Ende springt next wieder an den Anfang. Wichtig war `stopPropagation()` in den Button-Handlern, sonst bubbelt der Klick zum displayDIV und das Bild wird gleich wieder geschlossen.